



โครงสร้างข้อมูลกราฟและการพัฒนากราฟ (Graph data structure and Implementation)

อ.ลือพล พิพานเมฆาภรณ์

luepol.p@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

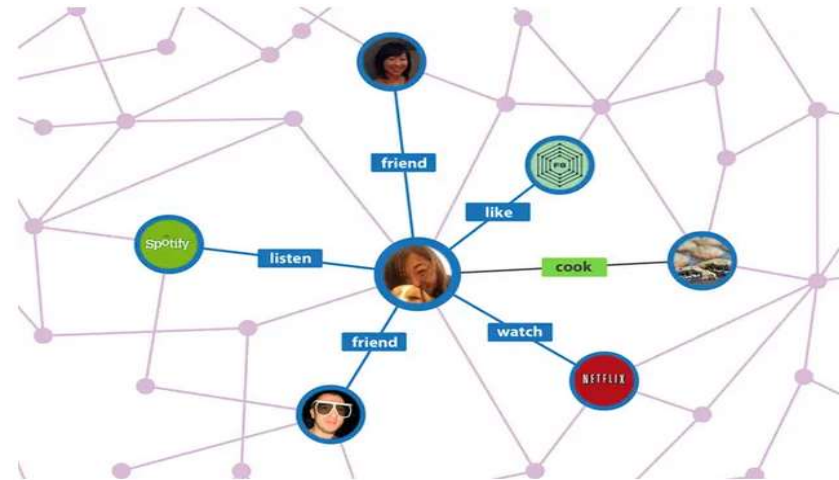
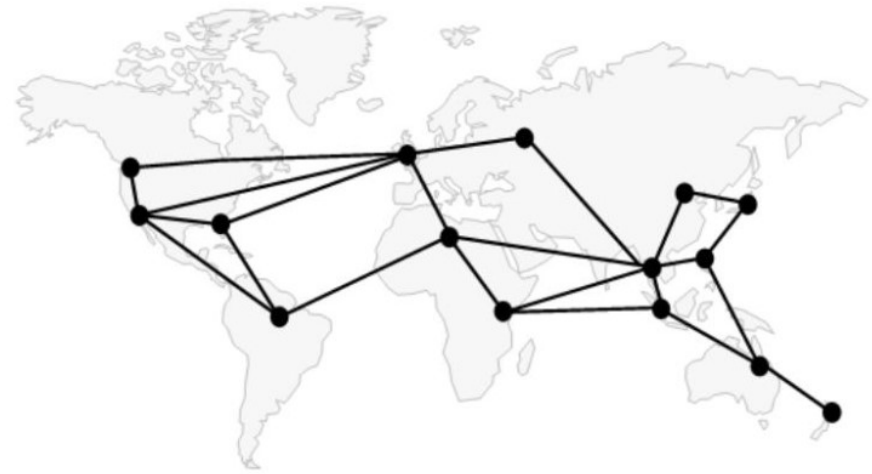
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ



กราฟ (graph)

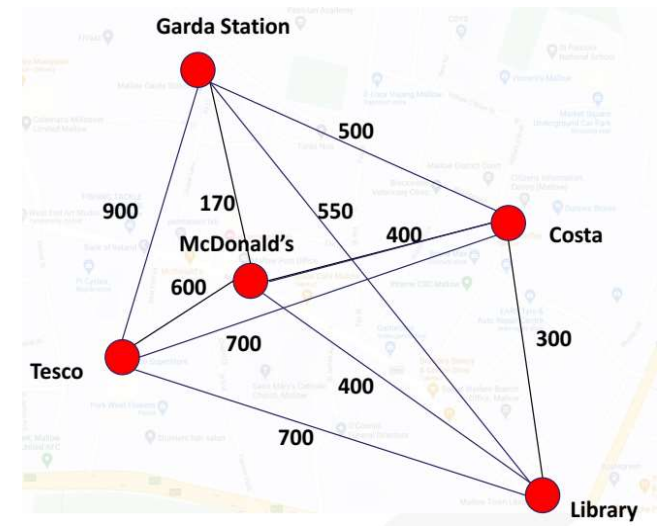
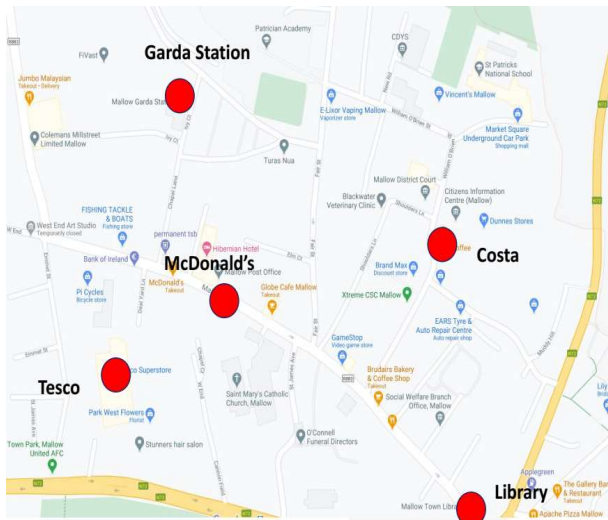
- กราฟ (graph) เป็นโครงสร้างข้อมูลที่ใช้แสดงความสัมพันธ์ระหว่างข้อมูลหรือเอนทิตี (entity) ที่สนใจ

- กราฟประกอบไปด้วย
 - เซตเวอร์เท็กซ์ (vertex)
 - เซตของเอจด์ (edge)





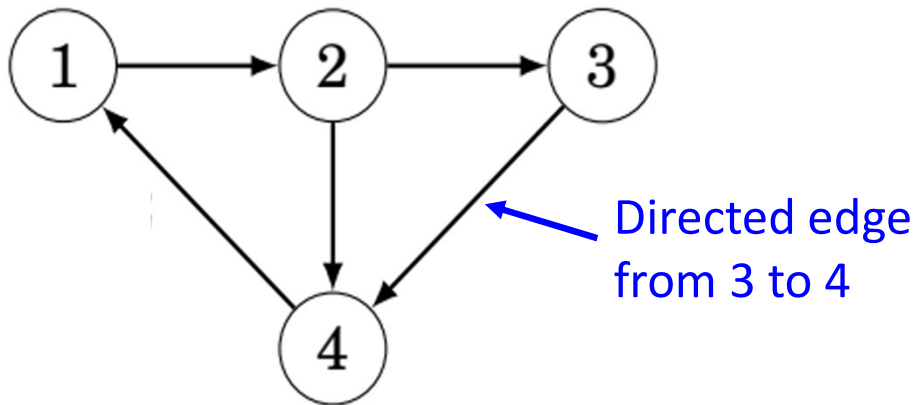
การจำลองปัญหาด้วยกราฟ



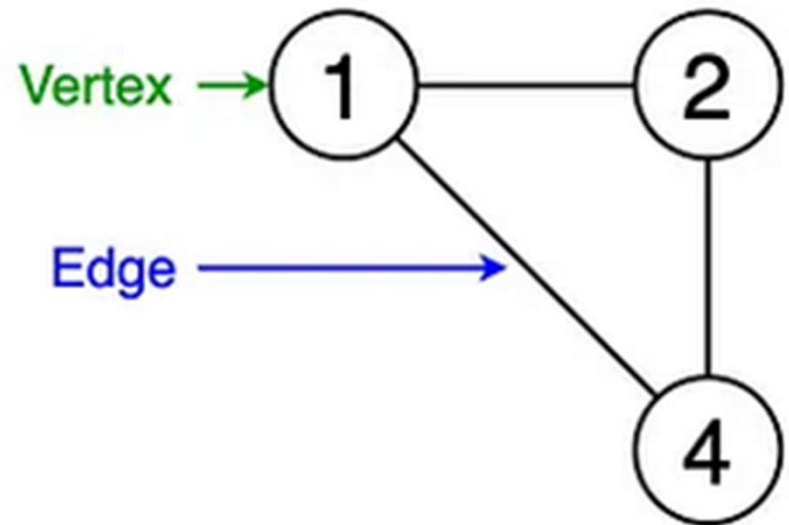
เวอร์เท็กซ์ (vertex) และเอดจ์ (edge)

■ กราฟ $G = (V, E)$ ถูกนิยามว่าเป็นเซตของเวอร์เท็กซ์ (vertex/vertices) และเอดจ์ (edge) โดยที่

- เวอร์เท็กซ์ ทำหน้าที่แทนเอนทิตี
- เอดจ์ คือความสัมพันธ์ระหว่างเวอร์เท็กซ์



กราฟทิศทาง



กราฟไม่มีทิศทาง



โครงสร้างข้อมูลกราฟ (Graph data structure)

- เมตริกซ์ประชิด (Adjacency Matrix)
 - มักใช้อาร์เรย์ 2 มิติ แทนเวอร์เท็กซ์ที่เชื่อมต่อกันด้วยเอดจ์
 - ข้อดีคือจัดการง่ายและทำงานได้รวดเร็ว
 - ข้อเสียคือสิ้นเปลืองหน่วยความจำ โดยเฉพาะอย่างยิ่ง sparse graph
- ลิสต์ประชิด (Adjacency List)
 - ใช้ลิสต์ร่วมกับอาร์เรย์ขนาด 1 มิติเพื่อเก็บเอดจ์ระหว่างเวอร์เท็กซ์
 - ข้อดีคือประหยัดหน่วยความจำที่ใช้ในการแสดงกราฟขนาดใหญ่
 - ข้อเสียคือยุ่งยากในการจัดการกว่าเมตริกซ์ประชิด

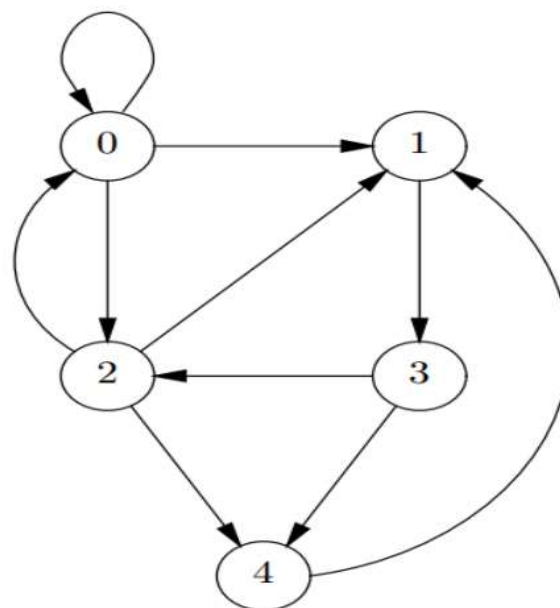
เมตริกซ์ประชิด (adjacency matrix)

เวอร์เท็กซ์

เวอร์เท็กซ์

	0	1	2	3	4
0	1	1	1	0	0
1	0	0	0	1	0
2	1	1	0	0	1
3	0	0	1	0	1
4	0	1	0	0	0

`int edges[5][5];`





เมตริกซ์ประชิด (adjacency matrix)

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n_vertex = 5;

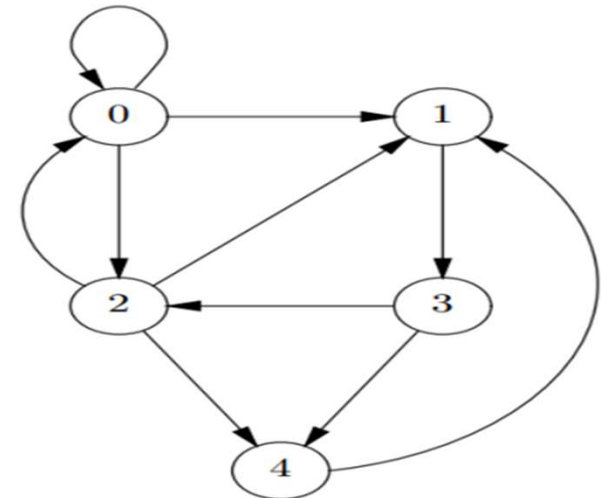
    // create 2D vector with n rows and n columns
    vector<vector<int>> graph(n_vertex, vector<int>(n_vertex, 0));

    int numVertices = graph.size(); // get no. rows

    graph[0][0] = 1; graph[0][1] = 1; graph[0][2] = 1;
    graph[1][3] = 1;
    graph[2][0] = 1; graph[2][1] = 1; graph[2][4] = 1;
    graph[3][2] = 1; graph[3][4] = 1;
    graph[4][1] = 1;

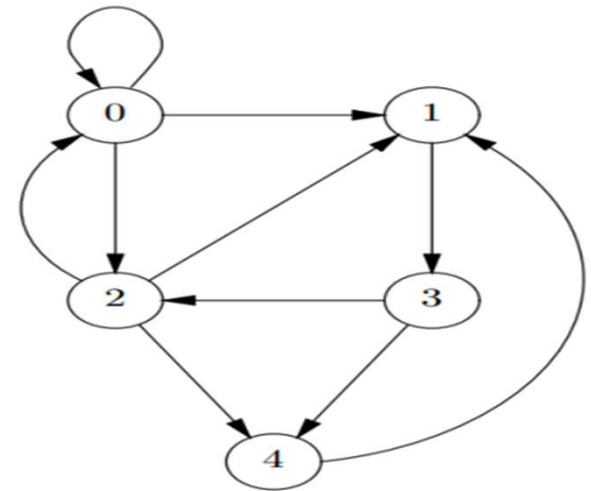
    print_graph(graph);

    return 0;
}
```



เมตริกซ์ประชิด (Adjacency matrix)

```
print_graph(vector<vector<int>>& graph)
{ for(int i=0; i < graph.size(); i++)
  { cout << i << " : ";
    for(int j=0; j < graph[i].size(); j++)
      cout << " " << graph[i][j];
    cout<<endl;
  }
}
```

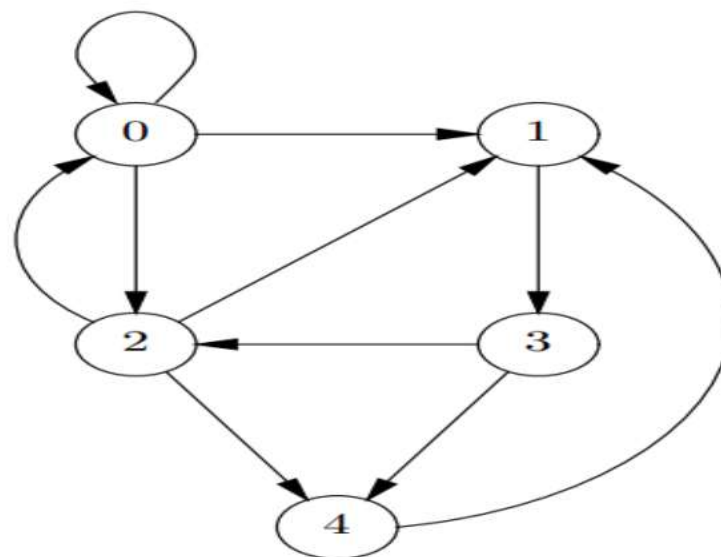
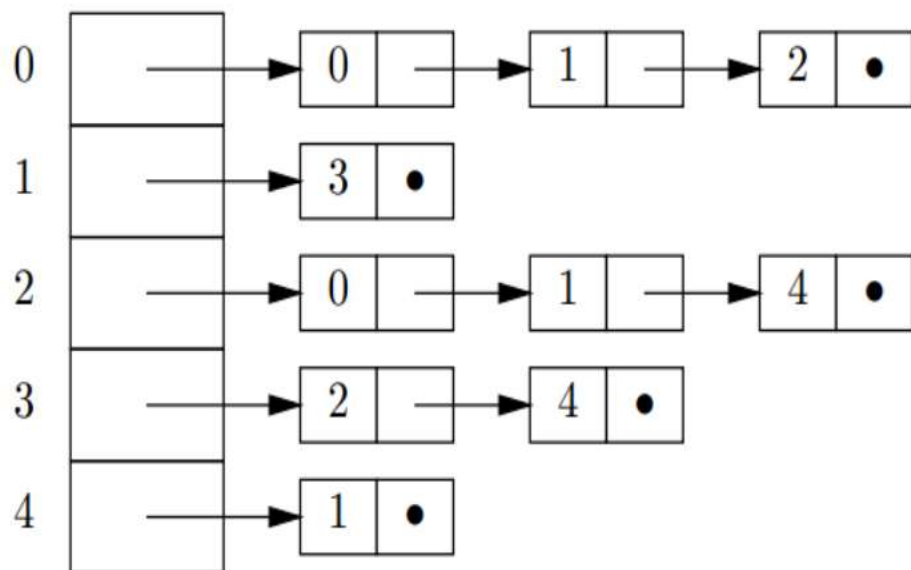


0	:	1	1	1	0	0
1	:	0	0	0	1	0
2	:	1	1	0	0	1
3	:	0	0	1	0	1
4	:	0	1	0	0	0

ลิสต์ประชิด (Adjacency list)

ประกอบไปด้วยอาร์เรย์ของลิสต์ของเอจด์ที่เชื่อมโยงในแต่ละเวอร์เทกซ์ของกราฟ

successors





ลิสต์ประชิดด้วยอาร์เรย์ของเวกเตอร์ (array of vector)

```
#include <iostream>
#include <vector>
```

```
using namespace std;
```

```
int main() {
    int n = 5;
    vector<int> adjList[n];

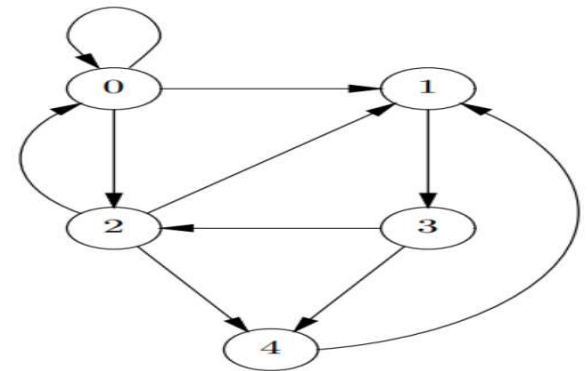
    // Adding edges to the graph
```

```
    adjList[0].push_back(0);    adjList[0].push_back(1);
    adjList[0].push_back(2);    adjList[1].push_back(3);
    adjList[2].push_back(0);    adjList[2].push_back(1);
    adjList[2].push_back(4);    adjList[3].push_back(2);
    adjList[3].push_back(4);    adjList[4].push_back(1);
```

```
    printGraph(adjList, n);
```

```
    return 0;
```

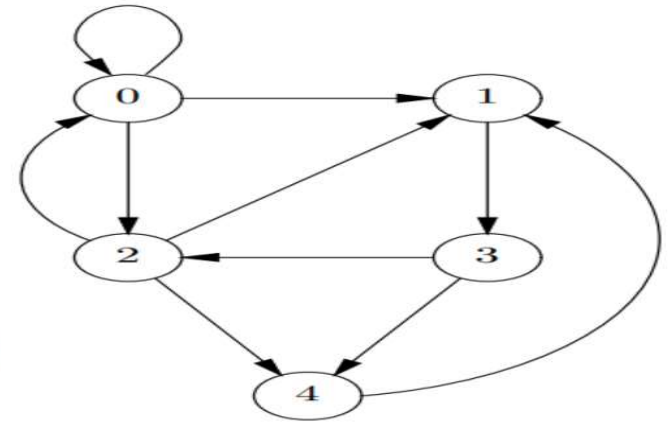
```
}
```



ลิสต์ประชิดด้วยอาร์เรย์ของเวกเตอร์ (array of vector)

```

void printGraph(vector<int> adjList[], int n) {
    for (int i = 0; i < n; ++i) {
        cout << i << ": ";
        for (int j=0; j<adjList[i].size(); j++) {
            cout << adjList[i][j] << " ";
        }
        cout << endl;
    }
}
    
```

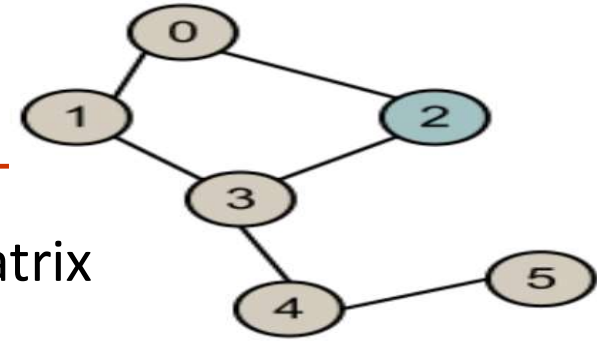


```

0: 0 1 2
1: 3
2: 0 1 4
3: 2 4
4: 1
    
```



แบบฝึกหัด



1. จงเขียนโปรแกรมเพื่อจัดเก็บกราฟด้านขวามือด้วยวิธี **adjacency matrix** พร้อมแสดงผลลัพธ์ในรูปแบบ **adj matrix**

2. จงเขียนโปรแกรมเพื่ออ่านข้อมูลเพื่อสร้างกราฟทิศทางด้วยวิธี **adjacency list** พร้อมแสดงผลลัพธ์ในรูปแบบ **adj. list** โดยที่

- บรรทัดแรก หมายถึง จำนวนเวอร์เท็กซ์ **n**
- บรรทัดสอง หมายถึง จำนวนเอดจ์ **m**
- **m** บรรทัดที่เหลือเป็นรายการเอดจ์ในกราฟ

input

5

6

1 2

2 3

3 4

4 5

4 6

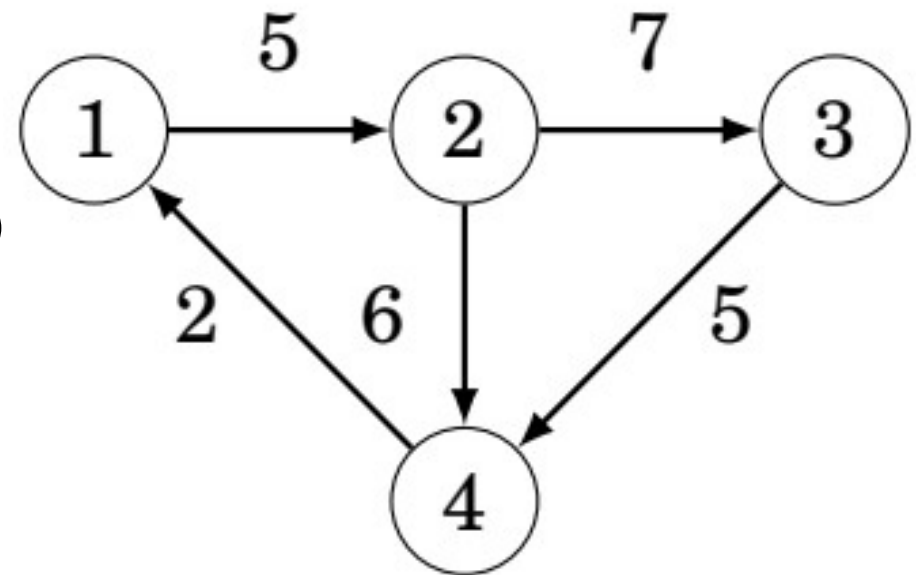
5 2

output

1:	2
2:	3
3:	4
4:	5 6
5:	2

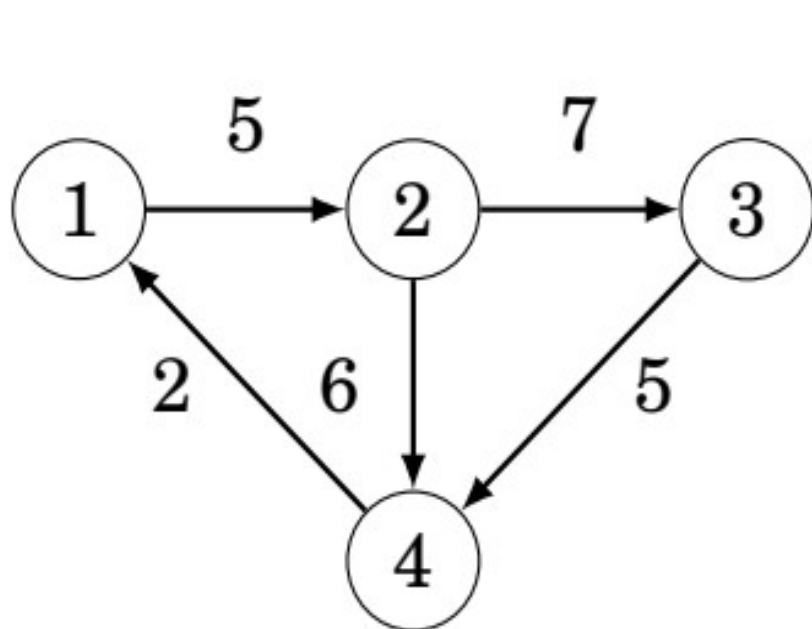
กราฟชนิดถ่วงน้ำหนัก (Weighted graph)

- กราฟชนิดถ่วงน้ำหนัก (weighted graph) เป็นรูปแบบกราฟที่มีการระบุค่าน้ำหนัก (weight) ให้กับเอจแต่ละเส้นในกราฟ
- ค่าน้ำหนักอาจแทนด้วยระยะทาง (distance) คะแนน (score) เวลา (time) หรือเงื่อนไขอื่นขึ้นอยู่กับการใช้งานกราฟ
- การแทนกราฟชนิดถ่วงน้ำหนักทำได้ 2 แบบ ได้แก่
 - เมตริกซ์ประชิด (adjacency matrix)
 - ลิสต์ประชิด (adjacency list)



เมตริกซ์ประชิด (Adjacency matrix)

```
vector<vector<int>> graph( n_vertex, vector<int>(n_vertex, 0) );
```



เวกเตอร์ที่เก็บ
เริ่มต้น

	1	2	3	4	เวกเตอร์ที่เก็บ ปลายทาง
1	0	5	0	0	
2	0	0	7	6	ค่าน้ำหนัก
3	0	0	0	5	
4	2	0	0	0	

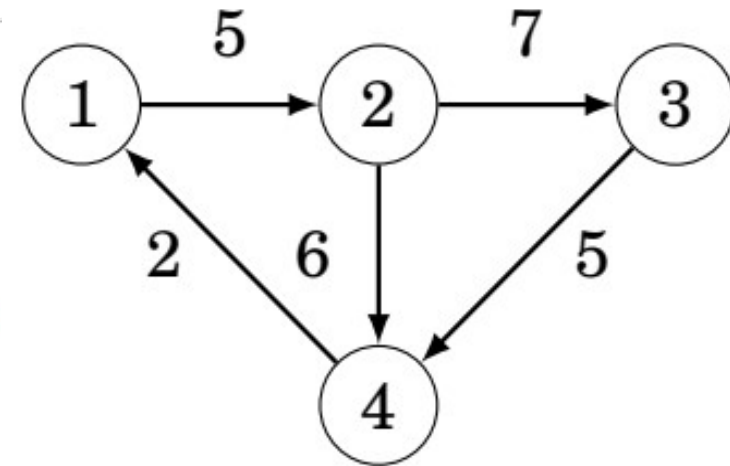
การเข้าถึงสมาชิกในเมตริกซ์ประชิด

```
#include <iostream>
#include <vector>

using namespace std;
int main() {
    int n_vertex = 4;
    vector<vector<int>> graph(n_vertex, vector<int>(n_vertex, 0));
    int numVertices = graph.size();

    graph[0][1] = 5; graph[1][2] = 7; graph[1][3] = 6;
    graph[2][3] = 5; graph[3][0] = 2;

    for(int i=0; i < graph.size(); i++)
    { cout << i << " : ";
      for(int j=0; j < graph[i].size(); j++)
          cout << " " << graph[i][j];
      cout<<endl;
    }
    return 0;
}
```



0	5	0	0
0	0	7	6
0	0	0	5
2	0	0	0



ลิสต์ประชิด (Adjacency list)

- สร้างอาร์เรย์ของเวกเตอร์ (vector) โดยสมาชิกแต่ละตัวจะแสดงในรูปแบบของคู่ลำดับ `pair<int, int>`
 - `pair.first` แทนเวอร์เท็กซ์ปลายทาง
 - `pair.second` แทนค่าน้ำหนัก (weight) ระหว่างเวอร์เท็กซ์

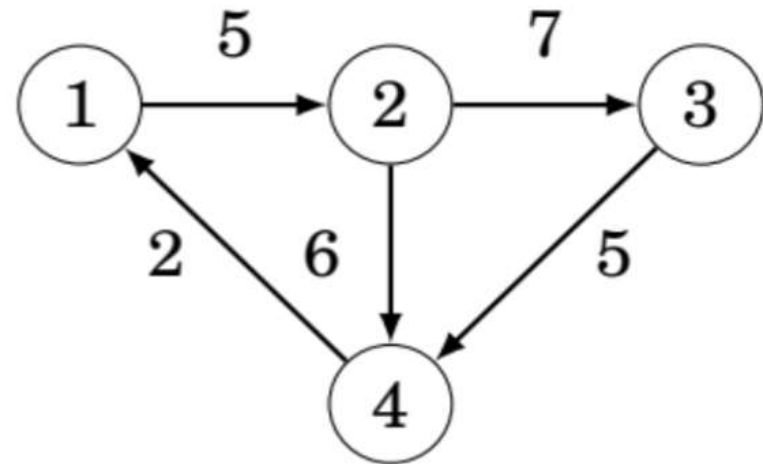
```
vector<pair<int,int>> adj[N];
```

<เวอร์เท็กซ์ปลายทาง, ค่าน้ำหนัก>

```
int main()
{
    vector<pair<int, int> > adj[4];

    adj[1].push_back({2, 5});
    adj[2].push_back({3, 7});
    adj[2].push_back({4, 6});
    adj[3].push_back({4, 5});
    adj[4].push_back({1, 2});

    return 0;
}
```





การเข้าถึงสมาชิกในลิสต์ประชิด

```
void print_graph(vector<pair<int, int>> adj[], int V)
{
    for(int i= 1; i <= V; ++i) {
        cout << i << ": ";
        for (int j = 0; j < adj[i].size(); ++j)
            cout << " (" << adj[i][j].first << ", " << adj[i][j].second << ") ";
        cout << "\n";
    }
}
```

```
1: (2,5)
2: (3,7) (4,6)
3: (4,5)
4: (1,2)
```